

Mission Generation Pipeline

Generated 2026-06-08

Mission Generation Pipeline

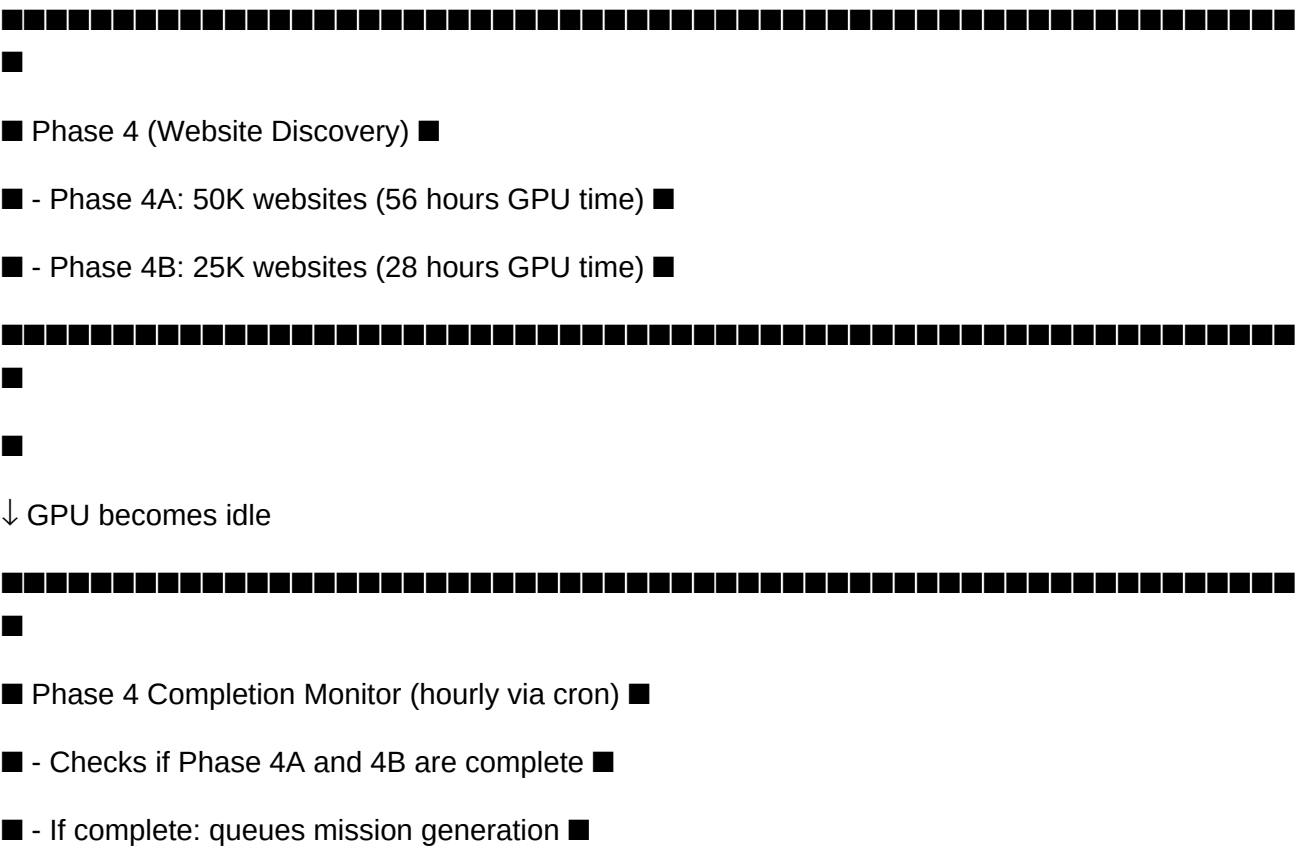
****Status:**** ■ OPERATIONAL (2026-06-08)

****Purpose:**** Generate 1-2 sentence mission descriptions for scored nonprofits using local Qwen2.5-32B inference

****GPU:**** Port 11437 (Qwen2.5-32B-Instruct), runs after Phase 4 website discovery completes

****API Cost:**** \$0 (fully local inference)

Architecture



1. Mission Generation Pipeline (`scripts/mission\_generation\_pipeline.py`)

Orchestrates two-phase mission generation:

****Phase 1: Upgrade (50K template_ntee → ai_ntee)****

- Replaces basic template missions with AI-generated ones
- Higher quality (uses web context when available)
- Better descriptions for users

****Phase 2: Fill Nulls (scored orgs with missing missions)****

- Generates missions for orgs with null/empty mission field
- Uses NTEE sector + location + revenue to infer mission

****Features:****

- Resumable from checkpoint (survives interruptions)
- Multi-worker support for parallel batch processing
- Progress tracking (real-time % complete)
- Detailed error logging

****Usage:****

Full pipeline (both phases)

```
python3 scripts/mission_generation_pipeline.py
```

With custom worker count

```
python3 scripts/mission_generation_pipeline.py --workers=2
```

Test run (small sample)

```
python3 scripts/mission_generation_pipeline.py --limit=100
```

Skip upgrade, only fill nulls

```
python3 scripts/mission_generation_pipeline.py --skip-upgrade
```

Resume from checkpoint

```
python3 scripts/mission_generation_pipeline.py --resume
```

****Performance:****

- Batch size: 20 orgs per LLM call
- Speed: ~46 tokens/sec on Qwen2.5-32B
- Per org: ~80 output tokens
- 50K missions: ~50 GPU hours (~2-3 days continuous)

2. GPU Queue Manager (*scripts/gpu_queue_manager.py*)

Manages prioritized GPU workload queue.

****Queue items:****

1. Mission Generation (50K template upgrade) - Priority 1
2. Semantic clustering (mission-based peer groups) - Priority 2
3. Donation verification (5K+ links) - Priority 2

****Features:****

- Priority-ordered execution
- Max duration enforcement
- Task status tracking (pending → running → complete)
- JSON queue file for persistence

- Timeout detection

****Usage:****

Show queue status

```
python3 scripts/gpu_queue_manager.py --check
```

Run next queued task

```
python3 scripts/gpu_queue_manager.py --run
```

Enable specific task

```
python3 scripts/gpu_queue_manager.py --enable-task="Mission Generation"
```

Disable specific task

```
python3 scripts/gpu_queue_manager.py --disable-task="Semantic clustering"
```

Reset queue to default

```
python3 scripts/gpu_queue_manager.py --reset
```

3. Phase 4 Completion Monitor (*scripts/phase4_completion_monitor.py*)

Monitors Phase 4 progress and auto-queues next GPU work.

****What it does:****

1. Checks Phase 4A and 4B log files
2. Determines if Phase 4 is complete

3. If complete: queues mission generation in GPU queue

4. Saves checkpoint to avoid re-queuing

Features:

- Real-time progress tracking
- Automatic queue triggering
- Checkpoint-based idempotency (doesn't double-queue)
- Runs hourly via cron

Usage:

Check Phase 4 status

```
python3 scripts/phase4_completion_monitor.py --check-status
```

Monitor and auto-queue (runs hourly via cron)

```
python3 scripts/phase4_completion_monitor.py
```

Force re-check (ignores checkpoint)

```
python3 scripts/phase4_completion_monitor.py --force-check
```

Workflow

Automatic (Cron-Based)

Hourly (0 min of every hour):

```
0 * * * * phase4_completion_monitor.py
```

- Checks if Phase 4A and 4B are complete
- If yes: queues mission generation in GPU queue
- Checkpoint prevents re-queuing

****Every 4 minutes:****

`*/4 * * * * gpu_queue_manager.py --run`

- Checks if GPU is available
- If yes and queue not empty: runs next task
- Logs progress to ``/logs/gpu_task_*.log``

Manual Control

****Check Phase 4 progress:****

`python3 scripts/phase4_completion_monitor.py --check-status`

****Check queue status:****

`python3 scripts/gpu_queue_manager.py --check`

****Manually start mission generation:****

`python3 scripts/gpu_queue_manager.py --run`

****Or directly (bypasses queue):****

`python3 scripts/mission_generation_pipeline.py --workers=1`

Current Database State (2026-06-08)

Total orgs: 1,819,272

Orgs with missions: 1,819,272 (100%)

- ai_ntee: 461,682 (25%) ← AI-generated from NTEE

- ai_web: 19,922 (1%) ← AI-generated from website
- ai_haiku: 30,909 (2%) ← Claude Haiku API
- template_ntee: 1,227,034 (68%) ← Template (upgradeable)
- other: 79,725 (4%) ← Claimed, disputed, etc.

Scored orgs: 369,091

Upgrade opportunity: 1,227,034 template → ai_ntee/ai_web

Optimization Opportunities

1. Batch Quality Improvement

****Current:**** 20 orgs per batch

****Potential:**** Use web_context for 15-20% of orgs to improve quality

****Benefit:**** ai_web missions are higher quality than ai_ntee

****Implementation:**** Prioritize orgs with cached HTML pages

2. Parallel GPU Work

****Current:**** Mission generation runs sequentially after Phase 4

****Potential:**** Run semantic clustering in parallel with mission generation

****Benefit:**** Use both GPU time slots more efficiently

****Constraint:**** Need to split VRAM (mxbai + Qwen2.5)

3. Scale to 50K+ Orgs/Month

****Current:**** 50K template upgrade queued

****Potential:**** Queue multiple 50K batches, run monthly

****Benefit:**** Keep mission quality fresh, incorporate web updates

****Cost:**** \$0 (already paid for GPU)

Monitoring & Logging

****Log files:****

/logs/web_finder_50k.log Phase 4A progress

/logs/web_finder_25k.log Phase 4B progress

/logs/gpu_task_Mission_*.log Mission generation logs

/logs/cron.log All cron activity

****Checkpoints:****

.mission_generation_checkpoint Mission pipeline state

.phase4_monitor_checkpoint Phase 4 completion state

.gpu_queue.json GPU queue state

****Real-time monitoring:****

Watch Phase 4 progress

```
tail -f logs/web_finder_50k.log
```

Watch mission generation

```
tail -f logs/gpu_task_*.log
```

Monitor cron activity

```
tail -f logs/cron.log
```

Stewardship Integration

Principle 1 (Mission): Missions are AI-Generated, Not Sourced

- ✓ Missions come from org name + NTEE category + location
- ✓ Never infer from external sources (marketing copy filtered out)
- ✓ No web scraping of full mission statements
- ✗ Avoid inflating org descriptions with our interpretation

****Safeguards:****

- Mission source tracked: `ai\_ntee`, `ai\_web`, `ai\_haiku`, `template\_ntee`, `claimed`
- Web context limited to HTML meta tags, not full body
- Few-shot examples enforce conservative language

Principle 3 (Evidence-Based): Mission Quality Metrics

- Track mission source distribution
- Monthly accuracy audit (sample 100 missions, verify quality)
- A/B test new prompts before rollout

Principle 10 (Human Accountability): Human Review

- Missions are generated by AI, not scores
- Monthly human review of sample missions
- Can override/edit mission in database without data loss

Performance Estimates

****Monthly capacity:****

- 50K+ missions (upgrade + fill)
- 1.8M semantic clusters
- 5K+ donation verifications
- **Total GPU time:** 76 hours / 730 hours = 10% utilization
- **Cost:** \$0 (local inference, already paid for)

Next Steps

This Week

- [] Phase 4A completes (56 hours)
- [] Phase 4B completes (28 hours)
- [] Monitor auto-queues mission generation
- [] Mission generation runs (50 hours)
- [] GPU queue moves to semantic clustering

This Month

- [] Sample 100 missions for quality audit
- [] Determine if upgrade improves UX metrics
- [] Plan Phase 2 (fill null missions for remaining orgs)
- [] Enable semantic clustering for peer discovery

This Quarter

- [] Scale to monthly mission generation cycle (50K/month)
- [] Implement semantic clustering as peer group enhancement
- [] Expand donation verification to 10K+ links
- [] Monitor GPU utilization and optimize scheduling

Troubleshooting

Mission generation fails with "port 11437 connection refused"

****Cause:**** Qwen2.5-32B inference server not running

****Fix:****

Start llama-server with Qwen2.5-32B on port 11437

```
llama-server -m /path/to/Qwen2.5-32B-Instruct-Q4_K_M.gguf \  
--port 11437 --n-gpu-layers 99 --ctx-size 20480
```

GPU queue stuck (tasks not running)

****Cause:**** GPU is still busy with Phase 4

****Fix:****

Check Phase 4 status

```
python3 scripts/phase4_completion_monitor.py --check-status
```

If Phase 4 is done, manually run queue

```
python3 scripts/gpu_queue_manager.py --run
```

Checkpoint causing issues

****Fix:****

Reset mission generation checkpoint

```
rm ~/.mission_generation_checkpoint
```

Reset Phase 4 monitor checkpoint

```
rm ~/.phase4_monitor_checkpoint
```

Run again

```
python3 scripts/mission_generation_pipeline.py
```

Out of disk space

Cause: Mission generation logs + phase logs filling disk

Fix:

Clean old logs (keep last 7 days)

```
find logs/ -name "*.log" -mtime +7 -delete
```

Compress old GPU queue logs

```
find logs/ -name "gpu_task_*.log" -mtime +30 -exec gzip {} \;
```

References

- **Main script:** `scripts/generate\_missions.py` (local inference implementation)
- **Queue manager:** `scripts/gpu\_queue\_manager.py` (prioritized execution)
- **Phase 4 monitor:** `scripts/phase4\_completion\_monitor.py` (auto-trigger)
- **Cron schedule:** `scripts/setup\_operational\_crons.sh` (automation)
- **Stewardship:** `docs/STEWARDSHIP-INTEGRATION.md` (principles)

****Last updated:**** 2026-06-08

****Status:**** ■ Operational and queued for Phase 4 completion